

Bioinformatics software development: Principles and future directions

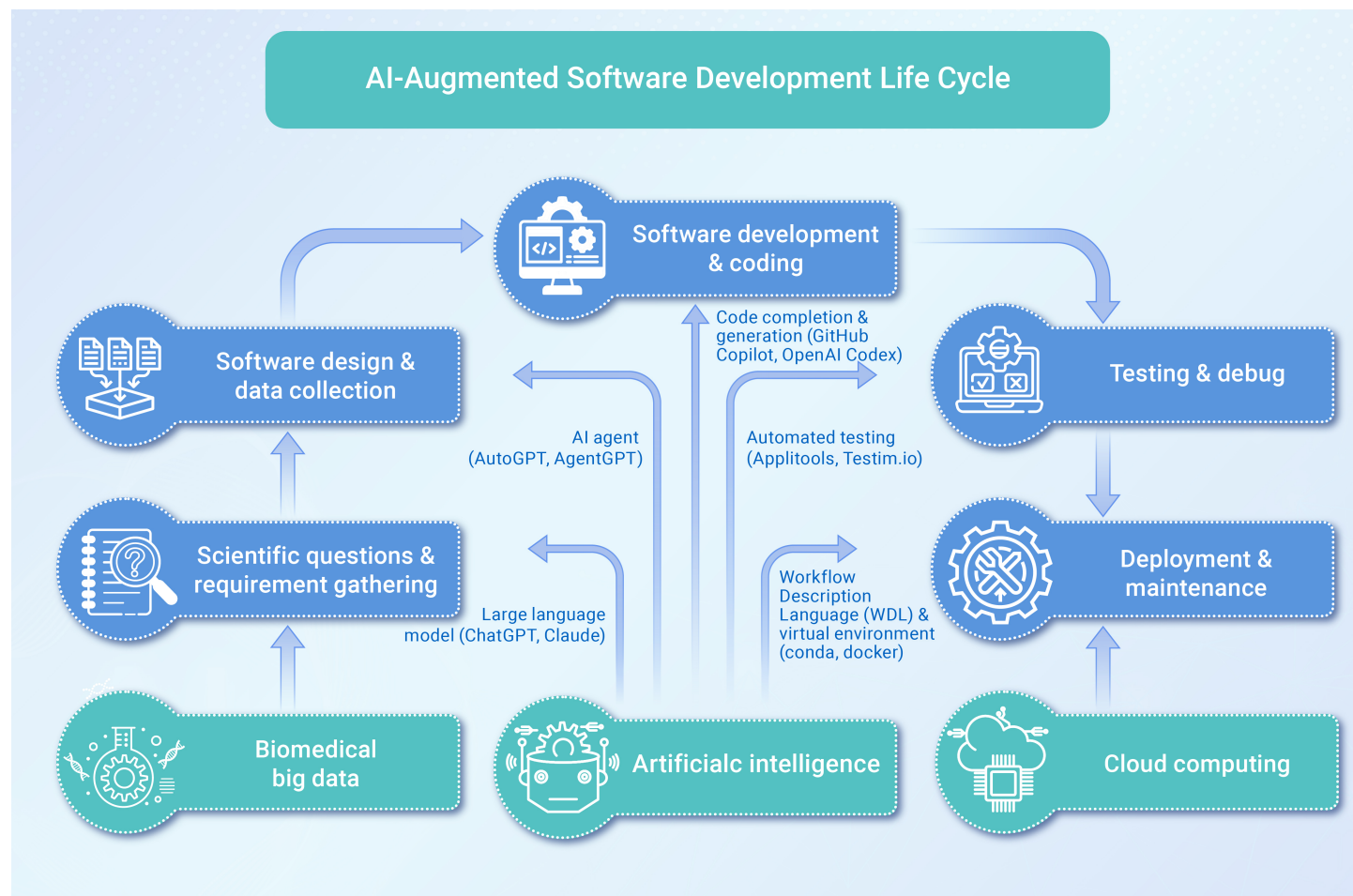
Xu-Kai Ma,^{1,9} Yan Yu,^{2,9} Tao Huang,^{3,9,*} Dake Zhang,⁴ Caihuan Tian,⁵ Wenli Tang,⁶ Ming Luo,⁷ Pufeng Du,^{8,*} Guangchuang Yu,^{6,*} and Li Yang^{1,*}

*Correspondence: huangtao@sibs.ac.cn (T.H.); liyang_fudan@fudan.edu.cn (L.Y.); gcyu1@smu.edu.cn (G.Y.); pdu@tju.edu.cn (P.D.)

Received: May 9, 2024; Accepted: July 21, 2024; Published Online: August 13, 2024; <https://doi.org/10.59717/j.xinn-life.2024.100083>

© 2024 The Author(s). This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

GRAPHICAL ABSTRACT



PUBLIC SUMMARY

- The growing volume of biomedical big data is driving a tremendous demand for advanced bioinformatics software.
- Artificial intelligence (AI) is expected to accelerate and enhance the development process of bioinformatics software.
- AI-augmented cloud computing is paving the way for the future of autonomous research.



Bioinformatics software development: Principles and future directions

Xu-Kai Ma,^{1,9} Yan Yu,^{2,9} Tao Huang,^{3,9,*} Dake Zhang,⁴ Caihuan Tian,⁵ Wenli Tang,⁶ Ming Luo,⁷ Pufeng Du,^{8,*} Guangchuang Yu,^{6,*} and Li Yang^{1,*}

¹Center for Molecular Medicine, Children's Hospital of Fudan University and Shanghai Key Laboratory of Medical Epigenetics, International Laboratory of Medical Epigenetics and Metabolism, Ministry of Science and Technology, Institutes of Biomedical Sciences, Fudan University, Shanghai 200032, China

²College of Life Sciences, Sichuan University, Chengdu 610065, China

³CAS Key Laboratory of Computational Biology, Shanghai Institute of Nutrition and Health, Chinese Academy of Sciences, Shanghai 200031, China

⁴Key Laboratory of Biomechanics and Mechanobiology, Ministry of Education, Beijing Advanced Innovation Center for Biomedical Engineering, School of Engineering Medicine, Beihang University, Beijing 100083, China

⁵Center of Peony, Institute of Vegetables and Flowers, Chinese Academy of Agricultural Sciences, Key Laboratory of Biology and Genetic Improvement of Flower Crops (North China), Key Laboratory of Biology and Genetic Improvement of Horticultural Crops, Ministry of Agriculture and Rural Affairs, Beijing 100081, China

⁶Department of Bioinformatics, School of Basic Medical Sciences, Southern Medical University, Guangzhou 510515, China

⁷South China Botanical Garden, Chinese Academy of Sciences, Guangzhou 510650, China

⁸College of Intelligence and Computing, Tianjin University, Tianjin 300350, China

⁹These authors contributed equally

*Correspondence: huangtao@sibs.ac.cn (T.H.); liyang_fudan@fudan.edu.cn (L.Y.); gcyu1@smu.edu.cn (G.Y.); pdu@tju.edu.cn (P.D.)

Received: May 9, 2024; Accepted: July 21, 2024; Published Online: August 13, 2024; <https://doi.org/10.59717/j.xinn-life.2024.100083>

© 2024 The Author(s). This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Citation: Ma X.-K., Yu Y., Huang T., et al., (2024). Bioinformatics software development: Principles and future directions. *The Innovation Life* 2(3): 100083.

The bioinformatics software for analyzing biomedical data is essential for converting raw data into meaningful biological insights. In this review, we outline the key stages and considerations in the development of bioinformatics software, using clusterProfiler and CIRCexplorer2 as illustrative examples. Furthermore, we examine some established large-scale life sciences platforms and summarize the design principles in the era of big data and Artificial Intelligence (AI) for open science. Future large-scale platforms are expected to offer graphical programming languages and transition from the sharing of data and codes to that of physical resources. The AI revolution will alter the landscape of bioinformatics software development and redefine the research paradigm of life sciences.

INTRODUCTION

Bioinformatics software has emerged as a cornerstone of modern biology, booming dramatically since the initiation of the Human Genome Project.¹ With advances in sequencing technology, the volume of biomedical data has exponentially expanded. The data storage size at NCBI (National Center for Biotechnology Information) offers a glimpse into this trend (Figure 1). To handle the ever-growing data and meet the diverse needs of the biomedical community, bioinformatics researchers need to continually develop and update the analysis software.

Over the past decades, numerous software programs have made substantial contributions to biology research. For instance, the Basic Local Alignment Search Tool (BLAST) is a fundamental component of bioinformatics pipelines.² As a standard tool for searching and aligning sequence data, BLAST powers various web front ends, including NCBI BLAST and Ensembl.^{3,4} Since its birth, BLAST has remained popular due to its simplicity and high sensitivity, while several novel sequence searching algorithms, such as DIAMOND and biological sequence analysis using profile hidden Markov models (HMMER),^{5,6} have been proposed to improve upon BLAST in terms of accuracy or speed. In contrast to BLAST, which is a flexible building block with a complex interface, Molecular Evolutionary Genetics Analysis (MEGA) focuses on providing a user-friendly experience for biologists.⁷ By providing an easy-to-use interface, MEGA enables users to perform data analyses, such as sequence alignment, phylogenetic tree reconstruction, and ancestral state reconstruction, without in-depth knowledge of algorithmic details.

Despite these advantages, utilizing software tools for bioinformatics analyses confronts with challenges as well. For example, the NCBI BLAST+ suite introduced an overly aggressive optimization for a commonly used parameter in 2012,⁸ which was not fixed until 2018 in BLAST+ 2.8.1.⁹ Such hidden software issues make results difficult to reproduce, or result in incorrect conclusions. Thus, there is a critical need for guidance in building user-friendly, interoperable, and reliable bioinformatics software. In the last decade, there has been a significant shift in data-driven science,¹⁰ fundamentally altering the landscape of software development. Biomedical big data have been generated at an unprecedented pace.^{11,12} Meanwhile, artificial intelligence

(AI) has made time and labor consuming jobs into automating tasks, such as understanding biological and clinical imaging data.¹³⁻¹⁶ The application of AI has also led to breakthroughs in previously insurmountable challenges such as protein folding and protein design.^{17,18} However, the processing capacity required for big data can be a barrier to accessibility, and the stochastic nature of deep learning algorithms complicates the interpretation and reproduction of results.¹⁹ In response, there is a need to shift from traditional software development models with *ad hoc* verification procedures to a new approach, which should emphasize rigorous validation, adherence to computer science theory, and the implementation of software engineering practices.²⁰

Efforts have been made to establish guidelines for bioinformatics data and software. The Findability, Accessibility, Interoperability, and Reusability (FAIR) Principles introduce a framework for managing scientific data with measurable goals,²¹ emphasizing the importance of curating data and metadata for easy discovery and reuse. The Global Alliance for Genomics and Health (GA4GH) has formulated technical standards and policy frameworks that enable efficient sharing of genomic data and collaboration on research using virtual resources.²² Despite these efforts, a gap remains between high-level guidelines and concrete work in bioinformatics software development. This review proposes principles of software development for life sciences, introduces essential resources and provides concrete examples to illustrate the implementation of these principles in the real-world application, with the aim of bridging the aforementioned gap and aiding the bioinformatics community in effective software development.

ESSENTIAL RESOURCES FOR SOFTWARE DEVELOPMENT

Bioinformatics software development is a multidisciplinary field that combines biology, computer science, and software engineering to analyze and interpret complex biological data. First of all, the development process relies on a wide range of essential resources, including comprehensive data repositories such as NCBI,²³ the European Bioinformatics Institute (EBI),²⁴ and the China National Center for Bioinformation (CNCB).²⁵ In addition, proficiency in programming languages like Python, R, or C++ is crucial, as they offer powerful capabilities for data manipulation, statistical analysis, and high-performance computing. Furthermore, open-source tools and libraries, including but not limited to BLAST, Biopython and Bioconductor facilitate collaboration and rapid software development by providing essential functionalities for sequence analysis, gene expression analysis, molecular docking, etc.^{26,27} Finally, community and collaboration platforms, such as GitHub, play a vital role in bioinformatics software development by allowing developers to share code, collaborate on projects, and contribute to open-source initiatives, which together promote a collaborative environment.

Public data repositories

In the process of software development, effectively utilizing public data resources is well accepted in the field. These public repositories are essential

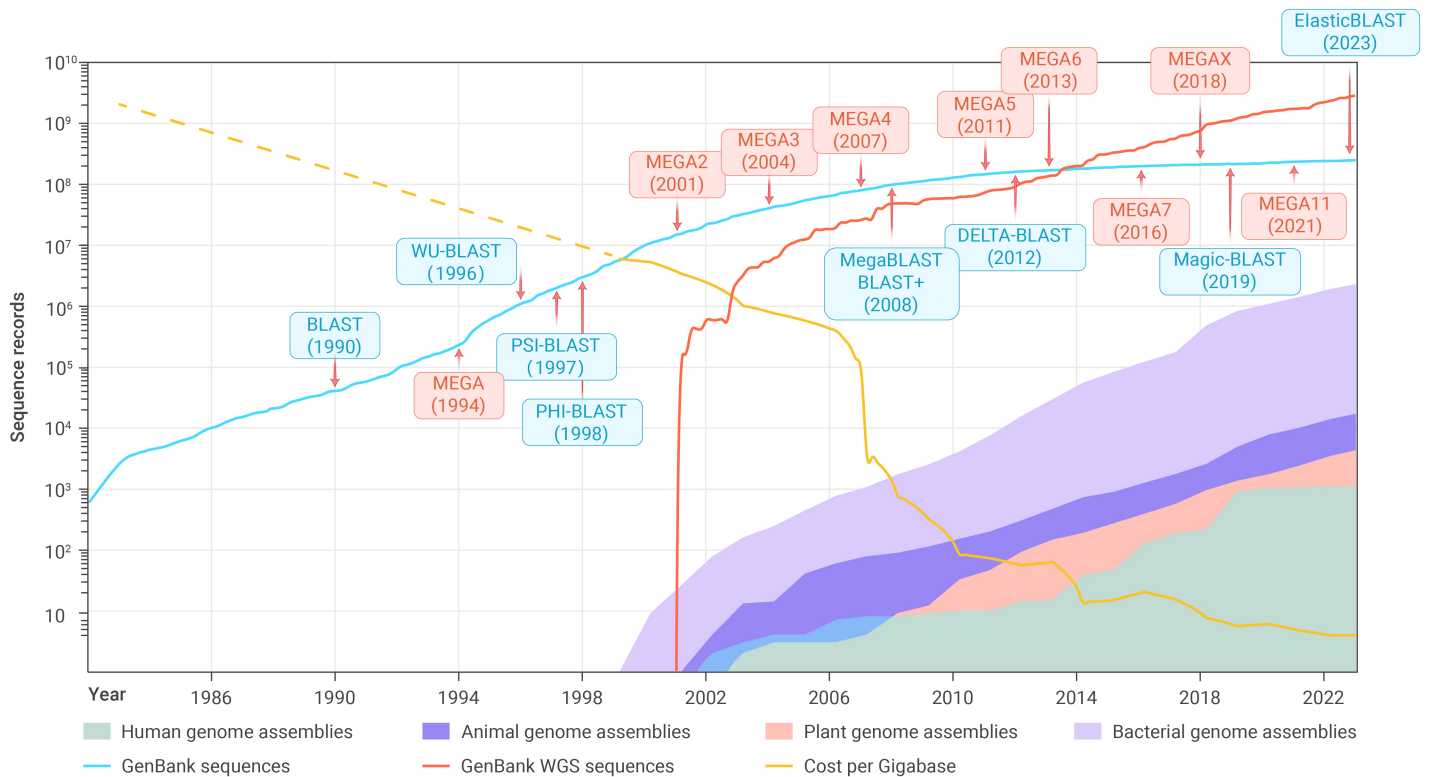


Figure 1. The growth of data at NCBI Decline in sequencing costs and advances of computation tools have led to a significant expansion of GenBank data. Major releases of BLAST and MEGA software are marked in their years of publication. Human, animal, plant and bacterial assemblies represent the number of genomic assemblies from the corresponding type of organism. GenBank sequences and GenBank WGS sequences denote the numbers of individually sequenced ('traditional') records and whole-genome sequencing records, respectively. Cost per Gigabase is measured in US dollars.

to the bioinformatics community, offering a wealth of data across various domains, including genomics, transcriptomics, proteomics, enzymatics, metabolomics, and *etc.* They significantly advance biomedical research by providing comprehensive datasets on different disciplines, including clinical data. These repositories not only facilitate data reuse, maximize research investments, and promote new discoveries, but also ensure data preservation in a standardized and accessible manner. Some main data repositories, such as NCBI, EBI, and CNCB, offer extensive resources that are crucial for the bioinformatics community.

The NCBI, part of the United States National Library of Medicine, is an important resource for molecular biology information. It houses a variety of databases, including Sequence Read Archive (SRA),²⁸ which is a publicly available repository that stores massive amounts of data from all branches of life, including metagenomic and environmental surveys, and GenBank,²⁹ which is a comprehensive collection of publicly available nucleotide sequences and their protein translations. Other notable databases from NCBI include RefSeq,³⁰ which provides reference sequences for genomes, genes and proteins, and ClinVar, which archives reports of the relationships among human genome variants and phenotypes.

The EBI, part of the European Molecular Biology Laboratory (EMBL), is another key player in the bioinformatics landscape. It offers a wide range of databases and tools, including the European Nucleotide Archive (ENA),³¹ which stores nucleotide sequence data, and UniProt,³² a comprehensive resource for protein sequence and functional information. EBI also maintains the Ensembl database,³³ which provides genome annotations for a wide range of species.

The CNCB is a leading institution in China dedicated to the collection, curation, and dissemination of bioinformatics data. CNCB provides access to a suite of databases, including Genome Sequence Archive (GSA),³⁴ which serves as a data repository designed to collect, manage, and disseminate raw sequence data, addressing the explosive growth of sequencing data volume and the increasing diversity of data types, and NODE database, which serves as a repository for a wide range of omics data types, including genomics, transcriptomics, proteomics and metabolomics.

These repositories, along with many others, such as the enzyme database BRENDA, and metabolomic databases like MetaboLights and the Human Metabolome Database (HMDB),³⁵⁻³⁷ provide essential data resources for bioinformatics software development. They ensure that researchers have access to high-quality, standardized data, which is crucial for developing reliable and effective bioinformatics tools. Some of bioinformatics databases are listed in Table S1. Researchers can find more databases from bioinformatics database indexes, such as Database Commons of CNCB, Online Bioinformatics Resources Collection (OBRC), bio.tools and biotresury.³⁸⁻⁴¹

Programming languages and open-source tools and libraries

The choice of programming languages and tools in bioinformatics is crucial for sure, as it can significantly impact the efficiency of data analyses.

Python has emerged as a dominant programming language in bioinformatics due to its simplicity, readability, and extensive library support. Biopython, a comprehensive Python library of bioinformatics tools, facilitates the manipulation and analyses of genomic and protein sequence data. Libraries such as Pandas, NumPy and scikit-learn further enhance Python's capabilities, enabling sophisticated data analyses and machine learning tasks.⁴²⁻⁴⁴ In addition, to package or use the software and libraries in a way that facilitates easy installation for end-users, developers can consider leveraging tools such as pip and conda (Figure 2).

The utilization of pip, a package manager for Python, allows users to install and manage libraries beyond the Python standard library. In order to distribute software via pip, it is essential for the software to be packaged and uploaded to the Python Package Index (PyPI, <https://pypi.org>). Conda is an open-source package management system and environment management system that is functional across Windows, macOS, and Linux. Conda packages are distributed through channels on Anaconda Cloud. Using CIRCexplorer2 as an example, the software can be installed using pip with 'pip install circexplorer2' or using conda with 'conda install bioconda:circexplorer2'.

R, another widely used language in bioinformatics, excels in statistical analyses and data visualizations. The Bioconductor project offers a vast array of tools for high-throughput genomic data analyses, making R indispensable

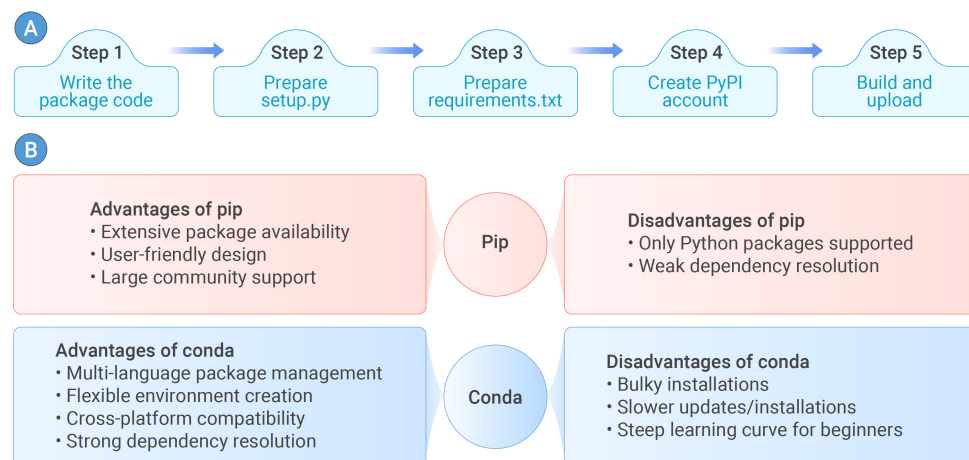


Figure 2. Comparison of the package managers pip and conda.

gral to advancing our understanding of complex biological systems and developing new bioinformatics methods.

The selection of programming languages and tools in bioinformatics is driven by the specific requirements of the project, including data type, analysis complexity, and performance needs. Python and R are favored for their extensive libraries and ease of use, while C/C++ and Java are chosen for performance-intensive tasks. Open-source tools and libraries are indispensable in enabling researchers to conduct advanced analyses

and promote innovation in the field of bioinformatics.

Of note, educational resources and literatures, including but not limited to online tutorials from platforms like the NCBI education page, provide foundational knowledge and practical guidances for developers, enabling them to understand the theoretical underpinnings and practical applications of bioinformatics tools.

Community and collaboration platforms

The swift progress of bioinformatics software relies on the cooperation of global bioinformatics researchers, which requires the backing of powerful code development collaboration platforms. GitHub (<https://github.com>) is one of the most important collaboration platforms in the software development world, offering project management and code sharing features. It enhances Git's functionality with an intuitive web interface, including features like access control, bug tracking, task management, continuous integration and project wikis (Figure 3). GitHub's ecosystem supports a vast array of programming languages and integrates with numerous tools and services, making it a versatile platform for developers worldwide.

Managing bioinformatics software on GitHub entails a series of procedures to ensure efficient development, maintenance and user involvement. These include using Git for version control with frequent, descriptive commits to track changes and facilitate collaboration, utilizing GitHub's issue tracking system to manage bug reports, feature requests and tasks, routinely releasing new versions of software, and adopting GitHub's release feature to tag and document each update (new features, bug fixes, and changes). Some bioinformatics software, such as the ImageGP webserver and ggClusterNet R packages,^{51,52} are released on GitHub prior to publication. This early release allows extensive user feedbacks, which significantly enhance the software's robustness and usability, facilitating a smoother publication process.

Gitee (<https://gitee.com>) is also a Git-based platform for code hosting and collaborative development. It offers similar functionalities to GitHub, such as source code hosting, pull requests for code review, issue tracking and wikis for project documentation. Gitee is optimized for the Chinese developer community, ensuring fast and reliable access that mitigates the connectivity issue Chinese developers frequently encountered when connecting to international services such as GitHub. SourceForge (<https://sourceforge.net>) provides free hosting and tools for open-source projects. It supports Git, Mercurial, and Subversion as version control systems. SourceForge is known for its comprehensive directory of open-source software projects, spanning various categories. There are other git-based platforms, such as GitLab (<https://about.gitlab.com>) and Bitbucket (<https://bitbucket.org>).

LIGHTWEIGHT R PACKAGE DEVELOPMENT

R, a programming language for data management, statistics, analysis, and visualization, has a rich ecosystem with over 20,000 R packages available on Comprehensive R Archive Network (CRAN) to date. These packages cover various disciplines such as ecology, epidemiology, finance, agriculture and etc. Bioconductor, based on the R language, is the largest open-source bioinformatics project (<https://bioconductor.org/>), with over 2,000 software pack-

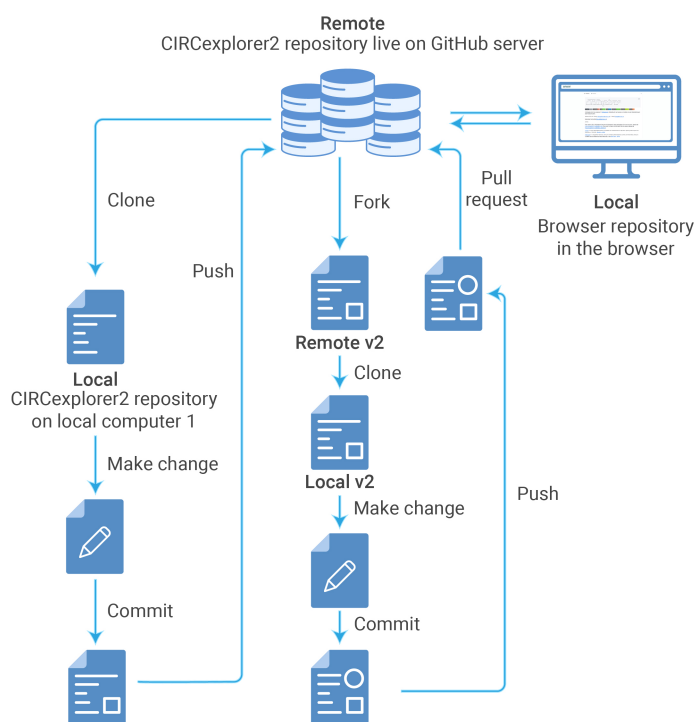


Figure 3. Diagram of the GitHub workflow.

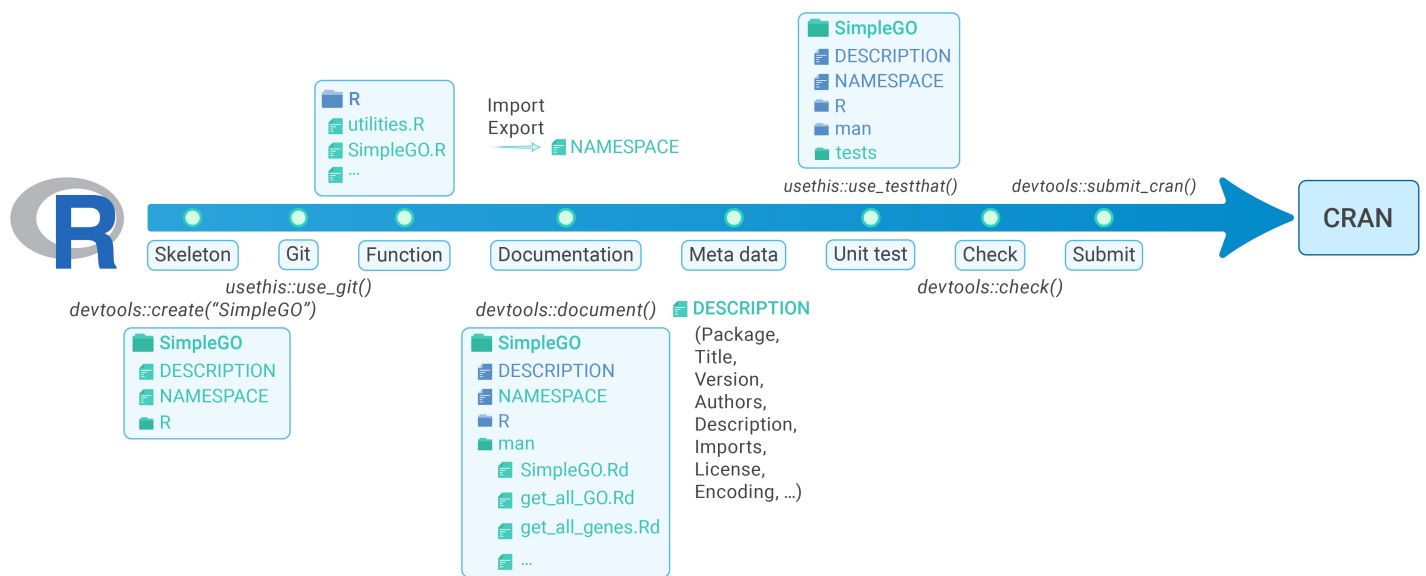
for researchers focusing on statistical genomics.²⁷ The ggplot2 package in R is renowned for its ability to create detailed and customizable visualizations, which are essential for interpreting complex biological data.⁴⁵

In addition to Python and R, other languages also have significant advantages in the bioinformatics software development in certain aspects. Perl, once a staple in bioinformatics, remains valuable for text processing and sequence analysis. Java is favored for developing large-scale and web-based bioinformatics applications due to its portability and robustness. C and C++ are preferred for performance-critical bioinformatics applications. MATLAB is utilized for numerical computing and algorithm development in bioinformatics.

Open-source tools also play a pivotal role in bioinformatics, providing researchers with the resources needed to perform sophisticated analyses. For example, BLAST is a fundamental tool in sequence analysis. Bowtie, an ultrafast and memory-efficient tool, is widely used for aligning sequencing reads to reference genomes.⁴⁶ Visualization tools like Genome Browsers (e.g., GBrowse and JBrowse) are essential for interpreting and presenting bioinformatics data.^{47,48} Workflow management systems such as Snakemake and Nextflow streamline the development of reproducible and scalable bioinformatics workflows, enhancing research efficiencies.^{49,50} These tools are inte-

Table 1. Useful packages for R package development

Package	Description
<i>covr</i>	Track and report code coverage
<i>devtools</i>	Collection of package development tools
<i>lintr</i>	Checks adherence to a given style, syntax errors and possible semantic issues
<i>rappdirs</i>	Determine which directories on the user's computer you should use to save data, caches and logs
<i>roxygen2</i>	Generate your Rd documentation, 'NAMESPACE' file, and collation field using specially formatted comments
<i>styler</i>	Pretty-prints R code without changing the user's formatting intent
<i>testthat</i>	Unit testing for R
<i>usethis</i>	Automate package and project setup tasks

Figure 4. The development of R package *simpleGO*.

ages specifically designed for the processing and visualization of omics data.²⁷ Of note, R language has an active community that continues to create new R packages, expanding its applications to diverse conditions and circumstances. In this scenario, it is useful to learn how to create an R package. As for personal requirement, organizing code into an R package, enhancing documentation, testing the code, and improving user-friendliness can greatly benefit future work. For instance, the creation of *clusterProfiler* initially aimed to compare enrichment analysis results from various cell cycle proteomics data.^{53,54} At that time, no tools supported the aggregation and comparison of results under multiple conditions. Similarly, the development of *ggtree* aimed to integrate and visualize phylogenetic trees and related data for joint presentation from an evolutionary perspective.⁵⁵ The eventual application scenarios exceeded the authors' initial expectations, which benefits others' research as well.

To start R package development, several necessary packages, including *usethis*, *roxygen2*, *testthat*, and *devtools*, are required (Table 1). Among them, *usethis* automates a series of settings for creating R packages; *roxygen2* supports literate programming, resulting in the direct embedding of documentation language in the code; *testthat* package is used to write test scripts to ensure the correctness of functions; *devtools* is a tool package for R development and management, which can simplify many development tasks.

We can use `devtools::create("package name")` to create the frame of an R package, which will generate the *DESCRIPTION* and *NAMESPACE* files and an *R* folder. *DESCRIPTION* is the core part of an R package, which includes the key metadata of the package. The *NAMESPACE* file contains information about exported and imported functions. The *R* folder is used to store source codes of the functions during software development, and functions are

usually stored in different files according to the logic of the R code. R code is the most critical because the main purpose of an R package is to write functions to solve specific problems or requirements. In addition to this, the R package will also include a *man* folder for storing documentation, containing package documentation, function documentation and data documentation.

Typically, it is recommended to use git, initialize by the `usethis::use_git()` function, for version control during software development. Running git remotely can establish a link between the local repository and the remote repository, allowing one to synchronize local development records with the remote repository, usually on a Git code hosting platform. Next, *clusterProfiler* is applied to illustrate the development process and standards of an R package that enables GO enrichment analysis.

The code in the R package needs to be defined in the form of functions and stored in files with the *.R* extension. Here, an illustrative package structure named *simpleGO* (<https://github.com/YuLab-SMU/simpleGO>) using `devtools::create("simpleGO")` is shown in Figure 4, which implements a simple gene ontology (GO) enrichment analysis function regarding *clusterProfiler*.

GO has three branches, Biological Process (BP), Molecular Function (MF), and Cellular Component (CC).⁵⁶ First, the auxiliary function `get_all_GO()` is defined to obtain all GO IDs belonging to a certain GO branch, and `get_all_genes()` is then defined to obtain all human genes annotated with the corresponding GO branch. Based on this, the main function `simpleGO()` is defined (<https://github.com/YuLab-SMU/simpleGO/blob/main/R/simpleGO.r>).

It accepts three inputs: *gene* is a vector of genes to be tested, such as differentially expressed genes. *GOID* is the selected GO ID by the user, and *background* is the total background genes for testing, usually all genes with

GO annotations, which can be the output of *get_all_genes()*. The *simpleGO()* function first determines the genes annotated by the GOID in the total background and the input gene vector, and then the necessary numeric information for calculating the *P*-value using the hypergeometric distribution can be determined. The function calculates the *P*-value and outputs the result as a data frame. Executing the *simpleGO()* function will test the enrichment level of the input gene vector under the given GOID. With this function, users only need a simple loop to test multiple or even all GO IDs which can be obtained from the *get_all_GO()* function.

Of note, after loading an R package or running a function, the working directory, output device, plot settings and other changes may affect the operation of other packages and functions. It is important to avoid such impacts. If a function must modify the environment settings, it should restore the original settings upon exit, which can be achieved using the *base::on.exit()* function. In this way, even the function encounters an error or exits prematurely, it will still execute the code to restore the original settings, thus avoiding any impact on the running environment. Additionally, the *withr* package can also be used to mitigate any effects on the running environment.

Proposed by Donald Knuth, literate programming is a programming method, which essentially combines code and documents.⁵⁷ In R language, this paradigm is supported by *roxygen2*. After writing the function code, the function document can be described in the form of code comments, and *roxygen2* will process the code comments to automatically generate a *NAMESPACE* file and *.Rd* files under the *man* folder. Here, we take the example of the *simpleGO()* function (<https://github.com/YuLab-SMU/simpleGO/blob/main/R/simpleGO.R>) and add *roxygen* code comments.

Roxygen2 comments start with *#'*, and each comment exists in the form of *@key value*. Among them, *@title* contains the name of the function. *@description* provides a description of the function. *@param* describes the inputs of the function with three inputs here. *@return* provides information about the function's output. *@examples* provides executable examples to demonstrate the usage of the function. *@export* indicates that this function needs to be included in the *NAMESPACE* for users to use, while *@importFrom* declares the function's dependencies, i.e., the functions called from other packages. This information will also be included in the *NAMESPACE*.

R will automatically generate help files under the *man* folder and create a new *NAMESPACE* file after running *devtools::document()*. One can use *?simpleGO* to retrieve the function's help documentation. Having code and documentation together has obvious benefits to remind developers to update the corresponding documentation.

Another important document is *DESCRIPTION*, which stores the metadata of the package. Every R package must have a *DESCRIPTION* file, which uses a simple file format called Debian control format (DCF). Each line contains a field and a value separated by a colon. The Title and Description fields describe what the package does. The Author field provides information about the author so that users know who to contact when there are issues with the package. The License field provides the copyright information for the package. Depends, Imports, and Suggests provide the package's dependencies.

Writing tests is highly recommended as it ensures that the function's functionality and output behave as expected. It has a positive impact on code development as well. For example, when dealing with a large function, simply testing the output may not guarantee that the internal logic is correct. Breaking down the large function into smaller functions allows for testing of the core elements of the function separately, ensuring their correctness. One can use *usethis::use_testthat()* to generate a testing environment, which will create a *tests* folder containing *testthat.R* and a *testthat* folder. Here, with a test file, *test-simpleGO.R* (<https://github.com/YuLab-SMU/simpleGO/blob/main/tests/testthat/test-simpleGO.R>), in the *testthat* folder, GO enrichment results can be compared between *simpleGO* and *clusterProfiler*, by evaluating whether the *p*-value returned by *simpleGO* is equal to that by *clusterProfiler*.

The above content covers the core aspects of R package development. Before releasing an R package, it is important to make sure it runs correctly. One can use *devtools::check()* to perform package checks, which will provide feedback on potential errors, warnings, and notes. Once the R package is confirmed without issues, the development of the R package is complete, and it can be released to the public.

R package can be published through code hosting platforms like GitHub, Gitee, GitLab, Bitbucket and SourceForge. For example, the *simpleGO* package is published on GitHub (<https://github.com/YuLab-SMU/simpleGO>). Publishing on CRAN requires some additional work, but it is worthwhile and recommended. Ultimately, it will increase the accessibility of the package to a larger user base because the CRAN platform tests whether the package can be run on different operating systems, compiles it into binary packages, and allows installation through the *install.packages()* function. Submitting an R package to CRAN is as simple as running the *devtools::submit_cran()* function, which creates the package bundle and submits it to CRAN using the author information (name and email) from the *DESCRIPTION* file, and notifies the developer to check for a confirmation email once the submission is successful.

LARGE SCALE OMICS SOFTWARE DEVELOPMENT

Before developing a bioinformatic software, it is essential to thoroughly comprehend the software's intended purpose, typically focused on resolving a scientific problem that satisfies a need within the research community. This process not only assists in defining the software's objectives but also guarantees that the ultimate result effectively meets the desired needs. When the goals are clearly defined, the software can be methodically designed and developed, contributing to the successful conclusion of the research project (Figure 5A).

Software design and implementation

Once the requirements are clear, the next step is to design the software architecture. This encompasses selecting the programming languages, algorithms and data structures to be utilized. The design phase should consider the efficiency, scalability and usability of the software, ensuring its capability to manage large datasets and provide a user-friendly interface (Figure 5B). The implementation phase encompasses the actual coding of the software, adhering to the design specifications. It is crucial to adopt good coding practices, such as composing clean, commented and modular code, to facilitate maintenance and future updates.

Selection of datasets

When developing software for next-generation sequencing (NGS) data analysis, the selection of the right NGS data is vital for the precise and efficient profiling of target molecules, such as circRNAs for CIRCexplorer2.⁵⁸ The choice of NGS data influences the software's ability to detect and characterize circRNAs, as different datasets offer variable levels of sensitivity and specificity (Figure 5C).⁵⁹ For instance, using nonpolyadenylated [poly(A)-] or rRNA depleted [ribo-] RNA-seq datasets, which contain nonpolyadenylated RNAs or include both polyadenylated and nonpolyadenylated RNAs, respectively, is more appropriate for circRNA profiling than using polyadenylated [poly(A)+] RNA-seq datasets. This is because circRNAs lack a poly(A) tail, thus fail to be enriched by poly(A)+ RNA-seq. Instead, they are included in poly(A)- and ribo- RNA-seq datasets.⁵⁹ Moreover, the use of RNase R-treated datasets can further enrich circRNAs, making them more detectable during sequencing and analysis. Various datasets can be obtained from publicly-available repositories as described above in the section of "Public Data Repositories".

Testing and evaluation

During developing computational tools for NGS data analysis, such as CIRCexplorer2,⁵⁸ the performance of testing extends basic functionality to guarantee accuracy, efficiency and scalability. This testing is vital for validating the biological precision of the software's results using established benchmarks, which is crucial for the precise detection and analyses of genetic elements like circRNAs. Furthermore, due to the extensive volume of NGS data, the software must efficiently process data to make the best use of computational resources and time, and to keep up with the increasing data output from newer NGS technologies without compromising performance.

To thoroughly evaluate NGS software performance, developers can employ a comprehensive approach that includes benchmarking for sensitivity and precision, as well as extensive testing of computational resource use, such as CPU and memory (Figure 5D). Scalability tests are also suggested to ensure

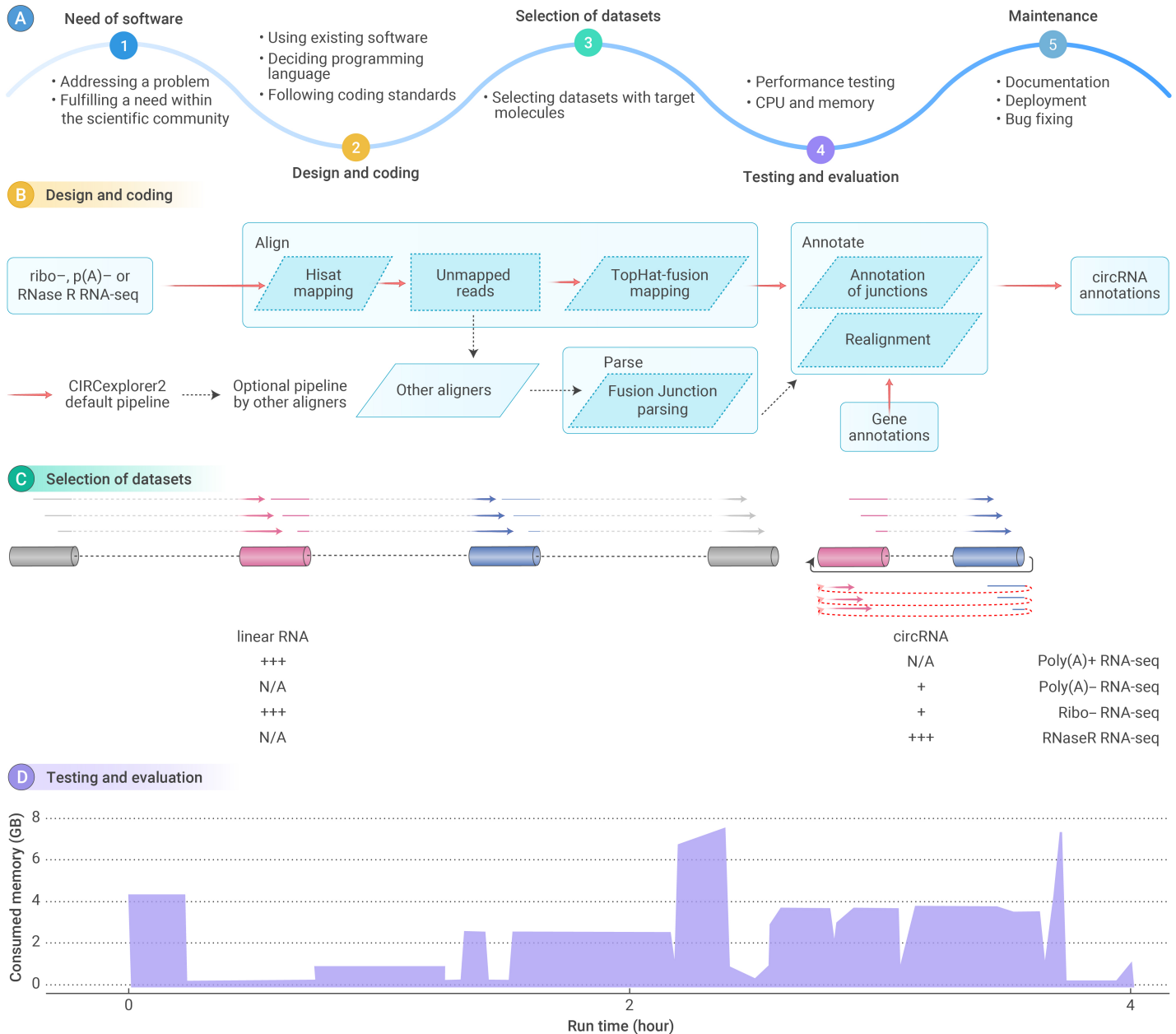


Figure 5. Development of software for analyzing NGS datasets (A) Diagram of the software development pipeline. (B) CIRCexplorer2 design diagram. (C) The circular RNAs (circRNAs) and linear RNAs that can be theoretically detected by different RNA-seq methods. (D) Diagram showing memory consumption in gigabytes (GB) during the execution of CIRCexplorer2.

the consistent performance of the software when running with different data volumes. Usability and reproducibility are examined to ensure that the tool is user-friendly and delivers consistent results across various runs and datasets.

Documentation, deployment and maintenance

Comprehensive documentation plays a crucial role in bioinformatics software, as it provides users with installation guides, usage examples and explanations of the output. Well-documented software significantly enhances its usability and adoption by the scientific community. Deployment makes the software available to end-users, which can be achieved by depositing them into repositories like GitHub, bioinformatics platforms like Bioconductor or containerization platforms like Docker.²⁷ Ongoing maintenance is essential after release to fix bugs and respond to users' feedbacks. Engaging with the user community via forums, mailing lists or social media can yield valuable insights into the software's usage and necessary improvements.

Employing CIRCexplorer2 as an example

Here, CIRCexplorer2 is selected to exemplify the software development process for NGS data analyses. CIRCexplorer2 was originally developed to identify and quantify circRNAs from next-generation sequencing data.⁵⁸ To identify circRNA-featured back-splicing sites, CIRCexplorer2 adopts existing tools like TopHat-Fusion or STAR for RNA-seq alignment to identify reads reversely mapped to back-splicing junction sites,^{60,61} scripted by Python programming language. Benchmark datasets were selected and downloaded from GEO database. The performance and accuracy of CIRCexplorer2 were tested (Figure 5C) and the scripts were iterated several times according to the evaluation results. Finally, CIRCexplorer2 was deployed on the GitHub (<https://github.com/YangLab/CIRCexplorer2>) and documented in great detail (<http://circexplorer2.readthedocs.org>). Issues and questions about CIRCexplorer2 can be asked through GitHub and by email. The CIRCexplorer series has undergone three major updates. Each major update serves to either broaden the software's capabilities, as seen in the transition from CIRCexplorer to CIRCexplorer2, or to resolve unique problems not addressed in

earlier versions, exemplified by the upgrade from CIRCexplorer2 to CIRCexplorer3/CLEAR.

OPEN SCIENCE ARCHITECTURE IN THE ERA OF AI

Over the past decade, cloud-based computing has significantly transformed various aspects of human life, including scientific research.^{62,63} Numerous cloud-based platforms have been developed for bioinformatics research. Cytoscape (<https://cytoscape.org>) was previously the standard for biological systems analysis and visualization, and has now evolved to incorporate NDEX (<https://www.ndexbio.org>),⁶⁴⁻⁶⁶ enhancing its capabilities for online analyses and visualizations of pathways.⁶⁷ Galaxy (<https://usegalaxy.org>) has been a cloud-native platform since its inception, continuously expanding its tools and workflows to address a broad range of genomics research challenges.⁶⁸ The UCSC Genome Browser (<https://genome.ucsc.edu>) is another example of leveraging cloud computing since its early days,⁶⁹ and it has now become a comprehensive platform for sharing and analyzing genomic data.⁷⁰ BioMart, one of the earliest platforms with web-oriented programming interfaces,⁷¹ is now a part of the Ensembl platform.³³ These platforms, which emerged in the first decade of the 21st century, have rapidly evolved with advancements in cloud computing.

Nowadays, cloud computing has revolutionized scientific collaboration by providing a common data set and mitigating the issues caused by unsynchronized data.⁷² Zenodo (<https://zenodo.org/>), a persistent cloud data storage for depositing scientific datasets, provides DOIs for each dataset. IEEE DataPort (<https://ieee-dataport.org/>) and Code Ocean (<https://codeocean.com/>) are solutions of IEEE for data and code sharing. Some IEEE journals and conferences now mandate the depositing of datasets and source codes using these services. Code Ocean not only acts as a platform for depositing source codes, but also supports code execution within customizable computational environments to ensure reproducibility of results.

These platforms have evolved beyond mere data and code sharing to become hubs for integrating collective human knowledge, embodying the principle of crowd intelligence.⁷³ They enable global collaboration, fostering innovation, new knowledge acquisition, and breakthroughs in science. Large-scale platforms for life sciences are typically built with a layered architecture (Figure 6).

To facilitate seamless communication between user-contributed components, these architectures implement comprehensive rules and standards, including data format specifications, communication protocols and security guidelines. This section aims to outline these common features as guidelines for the design and implementation of future large-scale platforms.

Establish user-aligned rules and standards before software development

The primary consideration in designing a large-scale life science architecture is to establish rules and standards that align with the expectations of potential users, prior to the actual development of software or code. These rules and standards encompass constraints that platform users are expected to adhere to, thereby enabling developers to create a system within a predefined framework. For instance, the UCSC genome browser defines the bigwig format for data tracking that users must follow. This design should precede the platform's development to ensure that it includes clear definitions of the data types.⁷⁴⁻⁷⁶ Another example for data format is the well-known FASTQ format,²⁶ which is commonly used in representing deep sequencing results. To some extent, this kind of data format standard itself is a platform that allows users to share data.^{77,78}

In terms of programming interfaces, adherence to existing rules and common practices is important. NCBI's E-utilities, for instance, is a well-established programming interface that allows users to programmatically query almost all records in NCBI databases.⁷⁹⁻⁸¹ This interface is provided through web, making it independent of specific platforms and programming languages. It also keeps very good backward compatibility in the last 20 years, thereby reducing maintenance efforts for users. Another example is BioMart, which offers a RESTful API accessible via simple command line tools and provides sample scripts to minimize programming efforts for users.⁸² Galaxy, known as a comprehensive platform for bioinformatics, provides its programming interface via the web and offers programming language bindings for Python, Java, PHP, and JavaScript, facilitating seam-

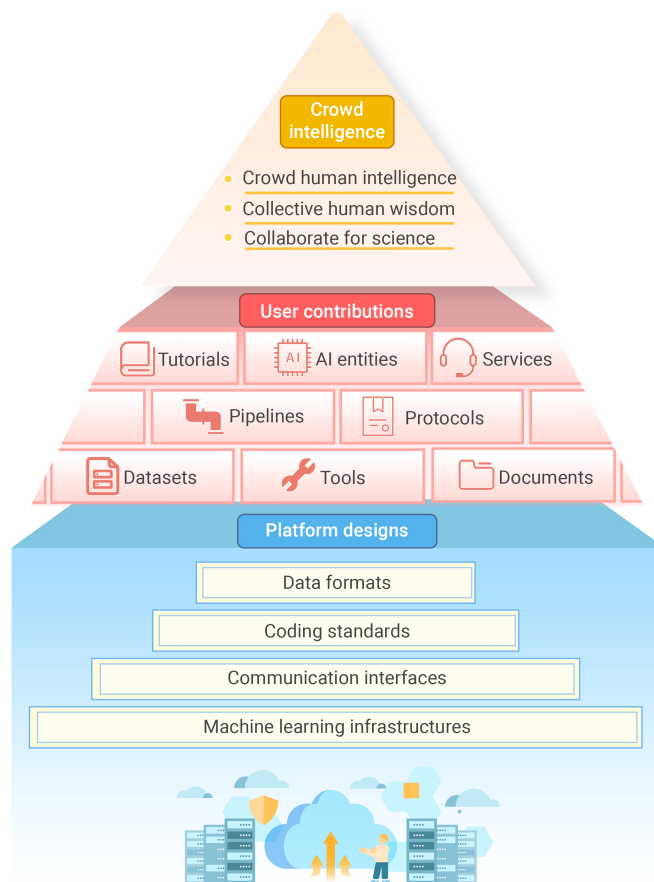


Figure 6. Global architecture of large-scale platform for open science All systems live in the cloud with a scalable infrastructure basis. The architecture sets rules and standards for codes, data and communications. It also provides a set of common routines that can be used for developing advanced applications. Users of the platform utilize the resources to create and share their datasets, codes, pipelines, and many other pieces for detailed advanced applications. All these applications can be used in a collaborative manner to solve scientific problems, with a crowd human intelligence.

less integration of user algorithms and codes.⁶⁸ All these programming interfaces are independent of user platforms, neutral regarding programming languages, web-based, and designed to facilitate learning. These principles are considered essential when designing programming interfaces.

Enable crowd intelligence in the cloud

The second principle in designing a large-scale architecture is to enable crowd intelligence in the cloud. Nowadays, most scientific studies are the result of collaborative efforts rather than the work of a single individual. Therefore, a platform should enable collaboration among people from different locations to work towards a common goal. By uniting human minds on the platform, a collective human wisdom — often referred to as 'crowd intelligence' — emerges. To achieve this, a large-scale platform should be built on cloud computation, which offers highly flexible and scalable computational power through the internet.

Taking the Galaxy as an example, it not only serves as a platform for data and program sharing but also provides resources for users to create their own tools for specific purposes.⁶⁸ Users can share their creations and collaborate on solving scientific questions. The platform operates in the cloud, with scalable computational resources through technologies like virtualization and containers. Galaxy also offers valuable resources such as training courses and tutorials contributed by users when they share their pipelines and tools. Despite being contributed by different users, these tools and pipelines adhere to the same rules and standards for code and data, ensuring interoperability across different platforms.

The development of large-scale platforms for science places great emphasis on privacy protection and data security, especially in cloud-based platforms. Access to some datasets must be restricted to ensure proper attribu-

tion to data creators, recover costs incurred in data production and protect the privacy of subjects. This is particularly true when human subjects are discussed. Patient identities must be anonymized before data sharing, often achieved by employing hashing algorithms combined with salting techniques to maintain machine identifiability while removing human-readable identities. Additionally, federated computation and federated learning are important technologies that enable distributed or cloud-based machine learning with privacy protection.^{83,84}

Taken together, as aforementioned, the large-scale open science architecture is a complex system that integrates not only software, but also various real-world resources and human resources, with the aim of addressing real-world scientific challenges.

Integrate AI in the platform

Another principle to highlight is the integration of AI in the platform. Advanced AI models like AlphaGo,⁸⁵ ChatGPT (<https://openai.com/chatgpt>), DALL-E (<https://openai.com/dall-e-2/>), Gemini (<https://gemini.google.com>) and Sora (<https://openai.com/sora/>), offer compelling user experiences. Including AI-based predictions as supplemental data basis is becoming popular. AlphaFold,¹⁷ which is developed by Google, predicts protein structures for most reviewed and predicted protein sequences. Over 200 million protein structures predictions were deposited in AlphaFold database to accelerate related studies. Although these structures may contain errors, they have been used in many cases where the errors do not significantly affect the results. A large-platform may provide these AI-based predictions as supplemental data basis. Nevertheless, scientists must be aware that these data have no consolidated experimental evidence in a physical world. Therefore, the results mainly derived from these data should be extensively verified by experiments.

In a large-scale platform, AI-based assistants or copilot features can help users create their own tools by providing intelligent suggestions and automating routine tasks. AI agents, such as AutoGPT, can provide hints and solutions to many academic problems. Automatic programmers, such as GitHub Copilot (<https://docs.github.com/en/copilot>) and Devin (<https://preview.devin.ai/>), can produce usable codes in many cases, with constructive inputs by users. Undoubtedly, a large AI harbor including AI functions like AutoGPT and GitHub Copilot, will accelerate scientific studies. Of special note, it is crucial to use them with caution, as they may provide results that seem similar to serious academic content but could potentially contain inaccuracies.^{86,87}

Create an open ecosystem

The final principle of designing the large platform is to allow users to construct the platform as an ecosystem together. Existing large-scale platforms have been developed over many years, with their functions contributed by numerous users. Similar to the cell phones we use, pipelines and tools in these platforms also constitute an ecosystem. Each user and each piece of code contribute to the platform, enhancing its stability and allowing it to evolve alongside scientific progress. Establishing an open ecosystem is particularly crucial for newly developed large-scale platforms. Recently, a bioinformatics-oriented cloud-based operating system Bio-OS (<https://bio-os.github.io/>) was released. By hosting an open-source application development contest, Bio-OS rapidly enhances its capabilities and user base, establishing itself as a burgeoning platform for bioinformatics research (<https://bio-os.github.io/activity/>).

PERSPECTIVES

Discussions above focus on open platforms for bioinformatics studies, where most bioinformatics specialists possess some programming knowledge. However, platform users may also include individuals with backgrounds in life science or clinical medicine, lacking the ability to write code. One potential solution is to provide graphical programming languages, such as Google Blockly (<https://developers.google.com/blockly/>) or Scratch. Although these graphical languages may currently be too simplistic for serious scientific computations, their potential for expansion could make them more applicable in research settings. For instance, the Orange (<https://orangedatamining.com>) data mining platform is a general-purpose software

that allows users to perform data mining tasks with a graphical programming language. Users can design their workflow by intuitive drag and drop operations, which can be used to perform actual scientific computations, such as single cell sequencing data analysis.⁸⁸ Integrating graphical programming languages into large-scale science platforms can enable more individuals to contribute to scientific work, leveraging the collective human wisdom of a broader community.

Alternatively, overcoming the programming skill barriers for individuals with limited programming experiences can be achieved through AI-based assistants. These assistants can produce usable scripts and code snippets based on inputs with natural language instructions. These codes can execute given tasks without further customization, making them particularly beneficial for those with basic programming knowledges.

The potential for future large-scale platforms includes a shift towards prioritizing the sharing of physical resources rather than data and code only. While the cost of common biochemical or genomics wet experiments has decreased, infrastructure and skilled operators are still necessary in most cases. This has led to the emergence of numerous businesses assisting university labs and research institutes. Such large-scale platforms should facilitate the sharing of standardized wet-lab protocols and provide access to shared laboratory facilities, enabling remote wet experiments in controlled environments through automatic machines or robots.⁸⁹⁻⁹¹ This advancement will greatly enhance the reproducibility of scientific results in future. Envision being able to open an app on your phone, design an experimental protocol with drag and drop features, have it validated by AI-based simulations, and then have a robot in a remote lab execute the experiments while you supervise the process via web-cameras. After paying for the experiment online, you could access your raw data in the cloud. While there are still many obstacles between this vision and the current reality, it remains a feasible direction for the future.

In summary, developing bioinformatics software is a dynamic and interdisciplinary process that integrates software engineering, biology, computer science and statistics to analyze and interpret biological data. The developer needs a deep understanding of the biological questions and has access to reliable data resources. The software varies widely, from lightweight, like an R or Python package tailored for specific tasks, to more complex NGS analysis software designed for broader applications. Nowadays, the cloud computing and AI technologies have revolutionized the development of bioinformatics software and contributed to build the Open Science Architecture. In addition, AI agents can help recommending analysis procedures, generate scripts and conduct software testing. In the future, researchers will find it easier and faster to advance their studies by generating customized bioinformatics software. A new era in bioinformatics software development is on the horizon.

REFERENCES

1. Lander, E.S., Linton, L.M., Birren, B., et al. (2001). Initial sequencing and analysis of the human genome. *Nature* **409**: 860–921. DOI: 10.1038/35057062.
2. Altschul, S.F., Gish, W., Miller, W., et al. (1990). Basic local alignment search tool. *J. Mol. Biol.* **215**: 403–410. DOI: 10.1016/s0022-2836(05)80360-2.
3. Johnson, M., Zaretskaya, I., Raytselis, Y., et al. (2008). NCBI BLAST: A better web interface. *Nucleic Acids Res.* **36**: W5–9. DOI: 10.1093/nar/gkn201.
4. Li, W., Cowley, A., Uludag, M., et al. (2015). The EMBL-EBI bioinformatics web and programmatic tools framework. *Nucleic Acids Res.* **43**: W580–584. DOI: 10.1093/nar/gkv279.
5. Buchfink, B., Reuter, K., and Drost, H.G. (2021). Sensitive protein alignments at tree-of-life scale using DIAMOND. *Nat. Methods* **18**: 366–368. DOI: 10.1038/s41592-021-01101-x.
6. Eddy, S.R. (2009). A new generation of homology search tools based on probabilistic inference. *Genome Inform.* **23**: 205–211. DOI: 10.1142/9781848165632_0019.
7. Tamura, K., Stecher, G., and Kumar, S. (2021). MEGA11: Molecular evolutionary genetics analysis version 11. *Mol. Biol. Evol.* **38**: 3022–3027. DOI: 10.1093/molbev/msab120.
8. Shah, N., Nute, M.G., Warnow, T., et al. (2019). Misunderstood parameter of NCBI BLAST impacts the correctness of bioinformatics workflows. *Bioinformatics* **35**: 1613–1614. DOI: 10.1093/bioinformatics/bty833.
9. Madden, T.L., Busby, B., and Ye, J. (2019). Reply to the paper: Misunderstood parameters of NCBI BLAST impacts the correctness of bioinformatics workflows. *Bioinformatics* **35**: 2699–2700. DOI: 10.1093/bioinformatics/bty1026.
10. Xu, Y., Liu, X., Cao, X., et al. (2021). Artificial intelligence: A powerful paradigm for scientific research. *The Innovation* **2**: 100179. DOI: 10.1016/j.xinn.2021.100179.

11. Huang, T., Lan, L., Fang, X., et al. (2015). Promises and challenges of big data computing in health sciences. *Big Data Research* **2**: 2–11. DOI: 10.1016/j.bdr.2015.02.002.
12. Wen, L., Li, G., Huang, T., et al. (2022). Single-cell technologies: From research to application. *The Innovation* **3**: 100342. DOI: 10.1016/j.xinn.2022.100342.
13. Falk, T., Mai, D., Bensch, R., et al. (2019). U-Net: Deep learning for cell counting, detection, and morphometry. *Nat. Methods* **16**: 67–70. DOI: 10.1038/s41592-018-0261-2.
14. Huang, T., Xu, H., Wang, H., et al. (2023). Artificial intelligence for medicine: Progress, challenges, and perspectives. *The Innovation Medicine* **1**: 100030. DOI: 10.59717/j.xinn-med.2023.100030.
15. Xun, D., Wang, R., Zhang, X., et al. (2024). Microsnoop: A generalist tool for microscopy image representation. *The Innovation* **5**: 100166. DOI: 10.1016/j.xinn.2023.100541.
16. Zhao, C., Guo, L., Dong, J., et al. (2021). Mass spectrometry imaging-based multi-modal technique: Next-generation of biochemical analysis strategy. *The Innovation* **2**: 100166. DOI: 10.1016/j.xinn.2021.100151.
17. Jumper, J., Evans, R., Pritzel, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature* **596**: 583–589. DOI: 10.1038/s41586-021-03819-2.
18. Huang, T. and Li, Y. (2023). Current progress, challenges, and future perspectives of language models for protein representation and protein design. *The Innovation* **4**: 100446. DOI: 10.1016/j.xinn.2023.100446.
19. Novakovsky, G., Dexter, N., Libbrecht, M.W., et al. (2023). Obtaining genetics insights from deep learning via explainable artificial intelligence. *Nat. Rev. Genet.* **24**: 125–137. DOI: 10.1038/s41576-022-00532-2.
20. Yang, A., Troup, M., and Ho, J.W.K. (2017). Scalability and validation of big data bioinformatics software. *Comput. Struct. Biotechnol. J.* **15**: 379–386. DOI: 10.1016/j.csbj.2017.07.002.
21. Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Sci. Data* **3**: 160018. DOI: 10.1038/sdata.2016.18.
22. Rehm, H.L., Page, A.J.H., Smith, L., et al. (2021). GA4GH: International policies and standards for data sharing across genomic research and healthcare. *Cell Genom.* **1**: 100029. DOI: 10.1016/j.xgen.2021.100029.
23. Sayers, E.W., Beck, J., Bolton, E.E., et al. (2024). Database resources of the National Center for Biotechnology Information. *Nucleic Acids Res.* **52**: D33–D43. DOI: 10.1093/nar/gkad1044.
24. Cantelli, G., Bateman, A., Brooksbank, C., et al. (2022). The European Bioinformatics Institute (EMBL-EBI) in 2021. *Nucleic Acids Res.* **50**: D11–D19. DOI: 10.1093/nar/gkab1127.
25. Bao, Y. and Xue, Y. (2023). From BIG Data Center to China National Center for bioinformation. *Genomics, Proteomics Bioinf.* **21**: 900–903. DOI: 10.1016/j.gpb.2023.10.001.
26. Cock, P.J., Fields, C.J., Goto, N., et al. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Res.* **38**: 1767–1771. DOI: 10.1093/nar/gkp1137.
27. Reimers, M. and Carey, V.J. (2006). Bioconductor: An open source framework for bioinformatics and computational biology. *Methods Enzymol.* **411**: 119–134. DOI: 10.1016/S0076-6879(06)11008-3.
28. Kodama, Y., Shumway, M., and Leinonen, R. (2012). The sequence read archive: Explosive growth of sequencing data. *Nucleic Acids Res.* **40**: D54–D56. DOI: 10.1093/nar/gkr854.
29. Sayers, E.W., Cavanaugh, M., Clark, K., et al. (2024). GenBank 2024 update. *Nucleic Acids Res.* **52**: D134–D137. DOI: 10.1093/nar/gkad903.
30. O'Leary, N.A., Wright, M.W., Brister, J.R., et al. (2016). Reference sequence (RefSeq) database at NCBI: Current status, taxonomic expansion, and functional annotation. *Nucleic Acids Res.* **44**: D733–D745. DOI: 10.1093/nar/gkv1189.
31. Yuan, D., Ahamed, A., Burgin, J., et al. (2024). The European nucleotide archive in 2023. *Nucleic Acids Res.* **52**: D92–D97. DOI: 10.1093/nar/gkad1067.
32. UniProt Consortium. (2023). UniProt: the universal protein knowledgebase in 2023. *Nucleic Acids Res.* **51**: D523–D531. DOI: 10.1093/nar/gkac1052.
33. Martin, F.J., Amode, M.R., Aneja, A., et al. (2023). Ensembl 2023. *Nucleic Acids Res.* **51**: D933–d941. DOI: 10.1093/nar/gkac958.
34. Wang, Y., Song, F., Zhu, J., et al. (2017). GSA: Genome sequence archive. *Genomics, Proteomics Bioinf.* **15**: 14–18. DOI: 10.1016/j.gpb.2017.01.001.
35. Schomburg, I., Chang, A., Hofmann, O., et al. (2002). BRENDA: A resource for enzyme data and metabolic information. *Trends Biochem. Sci.* **27**: 54–56. DOI: 10.1016/S0968-0004(01)02027-8.
36. Yurekten, O., Payne, T., Tejera, N., et al. (2024). MetaboLights: Open data repository for metabolomics. *Nucleic Acids Res.* **52**: D640–D646. DOI: 10.1093/nar/gkad1045.
37. Wishart, D.S., Knox, C., Guo, A.C., et al. (2009). HMDB: A knowledgebase for the human metabolome. *Nucleic Acids Res.* **37**: D603–610. DOI: 10.1093/nar/gkn810.
38. Ma, L., Zou, D., Liu, L., et al. (2023). Database Commons: A catalog of worldwide biological databases. *Genomics, Proteomics Bioinf.* **21**: 1054–1058. DOI: 10.1016/j.gpb.2022.12.004.
39. Chen, Y.B., Chattopadhyay, A., Bergen, P., et al. (2007). The Online Bioinformatics Resources Collection at the University of Pittsburgh Health Sciences Library System—a one-stop gateway to online bioinformatics databases and software tools. *Nucleic Acids Res.* **35**: D780–785. DOI: 10.1093/nar/gkl781.
40. Ison, J., Ienasescu, H., Chmura, P., et al. (2019). The bio. tools registry of software tools and data resources for the life sciences. *Genome Biol.* **20**: 164. DOI: 10.1186/s13059-019-1772-6.
41. Zhao, Q., Zhou, X., Wu, J., et al. (2024). BioTreasury: A community-based repository enabling indexing and rating of bioinformatics tools. *Sci. China Life Sci.* **67**: 221–229. DOI: 10.1007/s11427-023-2509-x.
42. McKinney, W. (2011). Pandas: A foundational Python library for data analysis and statistics. *Python for high performance and scientific computing* **14**: 1–9.
43. Van Der Walt, S., Colbert, S.C., and Varoquaux, G. (2011). The NumPy array: A structure for efficient numerical computation. *Comput. Sci. Eng.* **13**: 22–30. DOI: 10.1109/MCSE.2011.37.
44. Fabian, P. (2011). Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* **12**: 2825–2830.
45. Ginstet, C. (2011). ggplot2: Elegant graphics for data analysis. *Journal of the Royal Statistical Society Series A (Statistics in Society)* **174**: 245–246. DOI: 10.1111/j.1467-985X.2010.00676_9.x.
46. Langmead, B. and Salzberg, S.L. (2012). Fast gapped-read alignment with Bowtie 2. *Nat. Methods* **9**: 357–359. DOI: 10.1038/nmeth.1923.
47. Stein, L.D. (2013). Using GBrowse 2.0 to visualize and share next-generation sequence data. *Brief Bioinform.* **14**: 162–171. DOI: 10.1093/bib/bbt001.
48. Skinner, M.E., Uzilov, A.V., Stein, L.D., et al. (2009). JBrowse: A next-generation genome browser. *Genome Res.* **19**: 1630–1638. DOI: 10.1101/gr.094607.109.
49. Koster J. and Rahmann, S. (2018). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics* **34**: 3600. DOI: 10.1093/bioinformatics/bty350.
50. Di Tommaso, P., Chatzou, M., Floden, E.W., et al. (2017). Nextflow enables reproducible computational workflows. *Nat. Biotechnol.* **35**: 316–319. DOI: 10.1038/nbt.3820.
51. Chen, T., Liu, Y.X., and Huang, L. (2022). ImageGP: An easy-to-use data visualization web server for scientific researchers. *Imeta.* **1**: e5. DOI: 10.1002/imt2.5.
52. Wen, T., Xie, P., Yang, S., et al. (2022). ggClusterNet: An R package for microbiome network analysis and modularity-based multiple network layouts. *Imeta.* **1**: e32. DOI: 10.1002/imt2.32.
53. Yu, G., Wang, L.G., Han, Y., et al. (2012). clusterProfiler: An R package for comparing biological themes among gene clusters. *Omics* **16**: 284–287. DOI: 10.1089/omi.2011.0118.
54. Wu, T., Hu, E., Xu, S., et al. (2021). clusterProfiler 4.0: A universal enrichment tool for interpreting omics data. *The Innovation* **2**: 100141. DOI: 10.1016/j.xinn.2021.100141.
55. Yu, G., Lam, T.T., Zhu, H., et al. (2018). Two methods for mapping and visualizing associated data on phylogeny using ggtree. *Mol. Biol. Evol.* **35**: 3041–3043. DOI: 10.1093/molbev/msy194.
56. The Gene Ontology Consortium (2019). The Gene Ontology resource: 20 years and still going strong. *Nucleic Acids Res.* **47**: D330–d338. DOI: 10.1093/nar/gky1055.
57. Knuth, D.E. (1984). Literate programming. *Comput. J.* **27**: 97–111. DOI: 10.1093/comjnl/27.2.97.
58. Zhang, X.O., Dong, R., Zhang, Y., et al. (2016). Diverse alternative back-splicing and alternative splicing landscape of circular RNAs. *Genome Res.* **26**: 1277–1287. DOI: 10.1101/gr.202895.115.
59. Ma, X.K., Zhai, S.N., and Yang, L. (2023). Approaches and challenges in genome-wide circular RNA identification and quantification. *Trends Genet.* **39**: 897–907. DOI: 10.1016/j.tig.2023.09.006.
60. Kim, D. and Salzberg, S.L. (2011). TopHat-Fusion: An algorithm for discovery of novel fusion transcripts. *Genome Biol.* **12**: R72. DOI: 10.1186/gb-2011-12-8-r72.
61. Dobin, A., Davis, C.A., Schlesinger, F., et al. (2013). STAR: Ultrafast universal RNA-seq aligner. *Bioinformatics* **29**: 15–21. DOI: 10.1093/bioinformatics/bts635.
62. Koppad, S., B, A., Gkoutos, G.V., et al. (2021). Cloud computing enabled big multi-omics data analytics. *Bioinform. Biol. Insights* **15**: 11779322211035921. DOI: 10.1177/11779322211035921.
63. Griebel, L., Prokosch, H.U., Köpcke, F., et al. (2015). A scoping review of cloud computing in healthcare. *BMC Med. Inform. Decis. Mak.* **15**: 17. DOI: 10.1186/s12911-015-0145-7.
64. Otasek, D., Morris, J.H., Bouças, J., et al. (2019). Cytoscape automation: Empowering workflow-based network analysis. *Genome Biol.* **20**: 185. DOI: 10.1186/s13059-019-1758-4.
65. Shannon, P., Markiel, A., Ozier, O., et al. (2003). Cytoscape: A software environment for integrated models of biomolecular interaction networks. *Genome Res.* **13**: 2498–2504. DOI: 10.1101/gr.1239303.
66. Pillich, R.T., Chen, J., Churas, C., et al. (2023). NDEX IQuery: A multi-method network gene set analysis leveraging the Network Data Exchange. *Bioinformatics* **39**: btad118. DOI: 10.1093/bioinformatics/btad118.
67. Pillich, R.T., Chen, J., Rynkov, V., et al. (2017). NDEX: A community resource for sharing and publishing of biological networks. *Methods Mol. Biol.* **1558**: 271–301. DOI: 10.1007/978-1-4939-6783-4_13.
68. Galaxy Community (2022). The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2022 update. *Nucleic Acids Res.* **50**: W345–w351. DOI: 10.1093/nar/gkac247.
69. Lee, C.M., Barber, G.P., Casper, J., et al. (2020). UCSC Genome Browser enters 20th year. *Nucleic Acids Res.* **48**: D756–D761. DOI: 10.1093/nar/gkz1012.

70. Nassar, L.R., Barber, G.P., Benet-Pagès, A., et al. (2023). The UCSC Genome Browser database: 2023 update. *Nucleic Acids Res.* **51**: D1188–d1195. DOI: 10.1093/nar/gkac1072.
71. Kasprzyk, A. (2011). BioMart: Driving a paradigm change in biological data management. *Database* **2011**: bar049. DOI: 10.1093/database/bar049.
72. Langmead, B. and Nellore, A. (2018). Cloud computing for genomic data analysis and collaboration. *Nat. Rev. Genet.* **19**: 208–219. DOI: 10.1038/nrg.2017.113.
73. WANG, H., YU, Y., WANG, T., et al. (2023). Crowd intelligence paradigm: A new paradigm shift in software development. *Sci. Sin. Inf.* **53**: 1490. DOI: 10.1360/SSI-2023-0064.
74. Kent, W.J., Zweig, A.S., Barber, G., et al. (2010). BigWig and BigBed: Enabling browsing of large distributed datasets. *Bioinformatics* **26**: 2204–2207. DOI: 10.1093/bioinformatics/btq351.
75. Pohl, A. and Beato, M. (2014). bwtool: A tool for bigWig files. *Bioinformatics* **30**: 1618–1619. DOI: 10.1093/bioinformatics/btu056.
76. Retel, J.S., Poehlmann, A., Chiou, J., et al. (2024). A fast machine learning dataloader for epigenetic tracks from BigWig files. *Bioinformatics* **40**: btad767. DOI: 10.1093/bioinformatics/btad767.
77. Mason, C.E., Zumbo, P., Sanders, S., et al. (2010). Standardizing the next generation of bioinformatics software development with BioHDF (HDF5). *Adv. Exp. Med. Biol.* **680**: 693–700. DOI: 10.1007/978-1-4419-5913-3_77.
78. Dougherty, M.T., Folk, M.J., Zadok, E., et al. (2009). Unifying biological image formats with HDF5. *Commun. ACM* **52**: 42–47. DOI: 10.1145/1562764.1562781.
79. Mrozek, D., Małysiak-Mrozek, B., and Siążnik, A. (2013). Search GenBank: Interactive orchestration and ad-hoc choreography of Web services in the exploration of the biomedical resources of the National Center For Biotechnology Information. *BMC Bioinformatics* **14**: 73. DOI: 10.1186/1471-2105-14-73.
80. Nadkarni, P.M. and Parikh, C.R. (2012). An eUtils toolset and its use for creating a pipeline to link genomics and proteomics analyses to domain-specific biomedical literature. *J. Clin. Bioinforma.* **2**: 9. DOI: 10.1186/2043-9113-2-9.
81. Conford, B., Almsaeed, A., Buehler, S., et al. (2020). Tripal EUtils: A Tripal module to increase exchange and reuse of genome assembly metadata. *Database* **2019**: baz143. DOI: 10.1093/database/baz143.
82. Smedley, D., Haider, S., Ballester, B., et al. (2009). BioMart-biological queries made easy. *BMC Genomics* **10**: 22. DOI: 10.1186/1471-2164-10-22.
83. Bharadwaj, A. and Cormode, G. (2024). Federated computation: A survey of concepts and challenges. *Distrib. Parallel Databases* **42**: 299–335. DOI: 10.1007/s10619-023-07438-w.
84. Narmadha, K. and Varalakshmi, P. (2022). Federated learning in healthcare: A privacy preserving approach. *Stud. Health Technol. Inform.* **294**: 194–198. DOI: 10.3233/shiti220436.
85. Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017). Mastering the game of Go without human knowledge. *Nature* **550**: 354–359. DOI: 10.1038/nature24270.
86. Biever, C. (2023). ChatGPT broke the Turing test - the race is on for new ways to assess AI. *Nature* **619**: 686–689. DOI: 10.1038/d41586-023-02361-7.
87. Lan, L., Huang, T., Li, Y., et al. (2023). A survey of cross-lingual text classification and its applications on fake news detection. *WSARAI* **01**: 2350003. DOI: 10.1142/s2811032323500030.
88. Stražar, M., Žagar, L., Kokošar, J., et al. (2019). scOrange-a tool for hands-on training of concepts from single-cell data analytics. *Bioinformatics* **35**: i4–i12. DOI: 10.1093/bioinformatics/btz348.
89. Abolhasani, M. and Kumacheva, E. (2023). The rise of self-driving labs in chemical and materials sciences. *Nat. Synth.* **2**: 483–492. DOI: 10.1038/s44160-022-00231-0.
90. Ha, T., Lee, D., Kwon, Y., et al. (2023). AI-driven robotic chemist for autonomous synthesis of organic molecules. *Sci. Adv.* **9**: eadj0461. DOI: 10.1126/sciadv.adj0461.
91. Zhu, Q., Zhang, F., Huang, Y., et al. (2022). An all-round AI-Chemist with a scientific mind. *Natl. Sci. Rev.* **9**: nwac190. DOI: 10.1093/nsr/nwac190.

FUNDING AND ACKNOWLEDGMENTS

This work was supported by National Natural Science Foundation of China (31925011), and the Shanghai Municipal Science and Technology Commission (23JS1400300, 23DX1900102 and 23YF1407400). The funders had no role in study design, data collection and analysis, decision to publish or preparation of the manuscript.

AUTHOR CONTRIBUTIONS

X.K.M., L.Y., C.T. and M.L. were responsible for “Essential Resources for Software Development” and “Large Scale Omics Software Development” section. Y.Y. and D.Z. were responsible for “Introduction” section. T.H. organized and integrated all parts of content. G.Y. was responsible for “Lightweight R Package Development” section. P.D. and T.H. were responsible for “Open Science Architecture in The Era Of AI” section. D.Z. and T.H. were responsible for “Perspectives” section. X.K.M., T.H. and L.Y. integrated the contents and edited the manuscript. All authors contributed to the article and approved the submitted version.

DECLARATION OF INTERESTS

G.Y. is an Editorial Board member of The Innovation Life. He was blinded from reviewing or making final decisions on the manuscript. The article was subject to the journal's standard procedures, with peer review handled independently of this Editorial Board member and their research groups. The other authors declare that they have no conflicts of interest.

ETHICAL STATEMENT AND PATIENT CONSENT

Not applicable.

DATA AND CODE AVAILABILITY

The illustrative R package “simpleGO” is publicly available on GitHub at <https://github.com/YuLab-SMU/simpleGO>

SUPPLEMENTAL INFORMATION

It can be found online at <https://doi.org/10.59717/j.xinn-life.2024.100083>